POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

Laboratorio 3
OpenFlow – Regole Reattive

25.03.2026 | Software Defined Networking

Messaggi controllore

1

Recap

1.1

Messaggio Packet-In

- Usato dallo switch per inviare pacchetti al controllore quando l'azione specificata è Output verso la porta riservata CONTROLLER
- Il messaggio contiene tutto o parte del pacchetto, *incapsulato* in un messaggio OpenFlow

Messaggio Packet-Out

- Usato dal controllore per chiedere l'invio di un singolo pacchetto su una porta specifica
- Il messaggio contiene l'intero pacchetto *incapsulato* in un messaggio OpenFlow
- Il messaggio contiene anche una lista di azioni, tra cui un'azione di Output

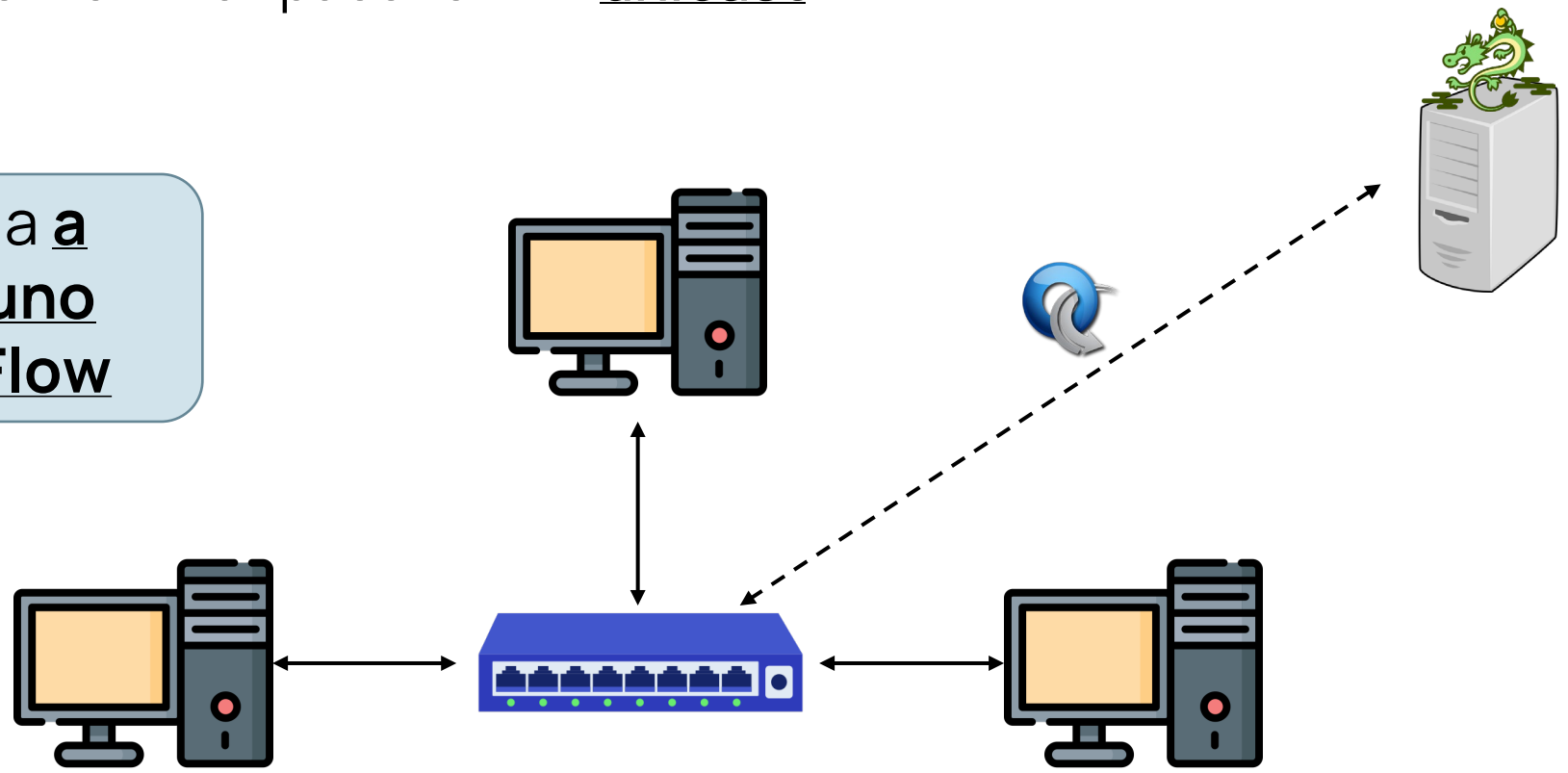
Hands-on (guidato)

1.2

Esercizio 1: Switch passando dal controllore (Packet-In e Packet-Out)

Realizziamo uno switch passando dal controllore. Il controllore apprende dove sono le stazioni e invia i pacchetti in ***unicast***.

Data una topologia **a stella**, realizzare **uno switch con OpenFlow**

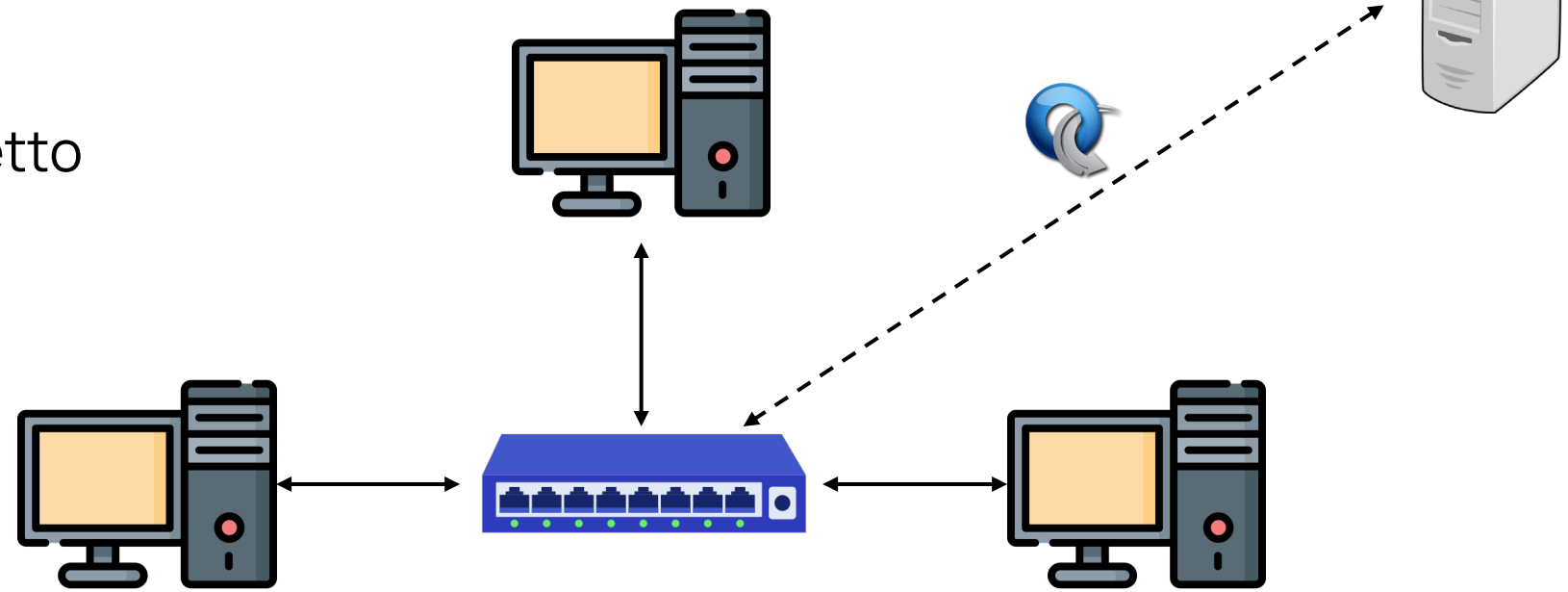


Esercizio 1: Switch passando dal controllore (Packet-In e Packet-Out)

- Quale regola inserire?
  - **Match -> ANY**
  - **Action -> CONTROLLER, NO_BUFFER**

In questo modo, mandiamo al controllore l'intero pacchetto

Data una topologia **a stella**, realizzare **uno switch con OpenFlow**

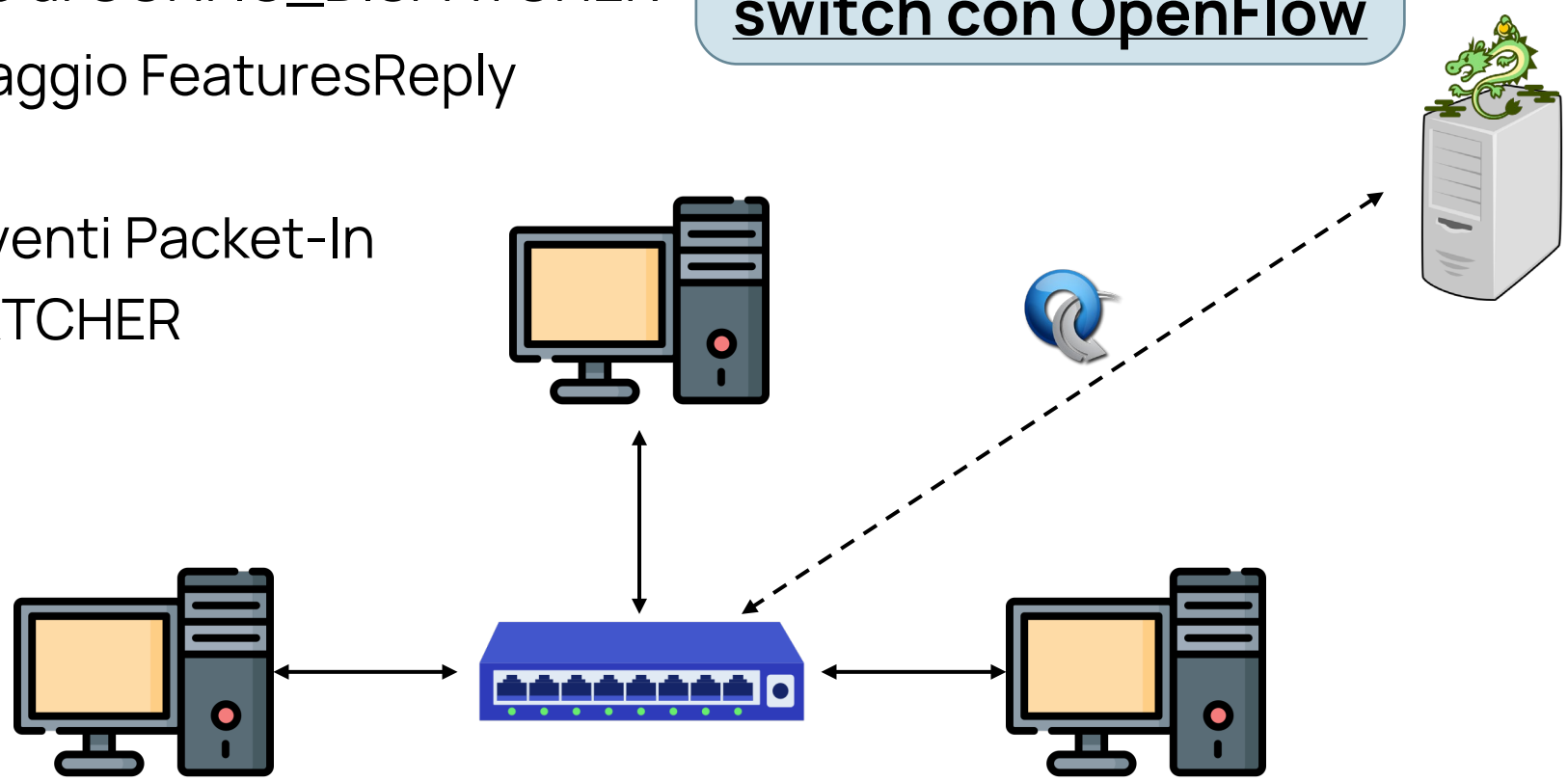


Esercizio 1: Switch passando dal controllore (Packet-In e Packet-Out)

- Quando inserire la regola?
  - Appena possibile -> fase di CONFIG_DISPATCHER alla ricezione del messaggio FeaturesReply

Data una topologia **a stella**, realizzare **uno switch con OpenFlow**

Inoltre, dobbiamo gestire gli eventi Packet-In durante la fase di MAIN_DISPATCHER



Big picture del nostro programma

import

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import set_ev_cls, CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.ofproto import ofproto_v1_3
```

Ryu Application

```
class PolimiSwitch(app_manager.RyuApp):
        OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
```

Costruttore

```
    def __init__(self, *args, **kwargs):
            super(PsrSwitch, self).__init__(*args, **kwargs)
            self.mac_to_port = {}
```

Features Handler

```
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
            ...
```

Packet-In Handler

```
    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
            ...
```

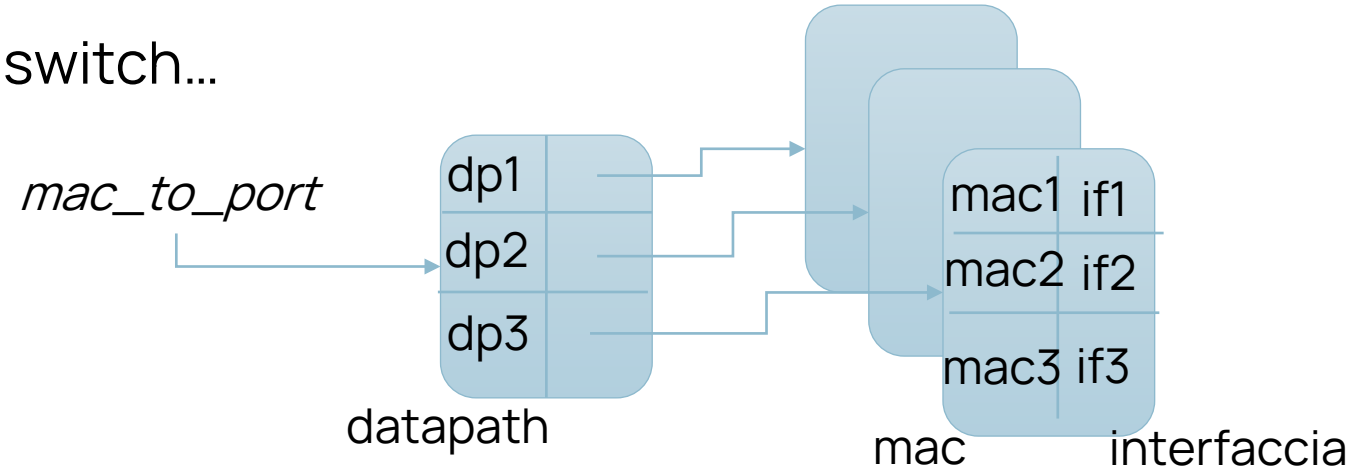
Costruttore

```
def __init__(self, *args, **kwargs):
        super(PsrSwitch, self).__init__(*args, **kwargs)
        self.mac_to_port = {}
```

Dizionario (vuoto)

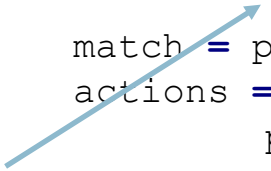
Ci serve una variabile che mantenga il suo valore tra eventi successivi. Ci serve una variabile di oggetto e un costruttore.

mac_to_port deve poter gestire tanti switch…
…quindi è un **dizionario di dizionari**!



Switch Features Handler

Aggiunge il datapath al dizionario



```
self.mac_to_port[datapath.id] = {}

match = parser.OFPMatch()
actions = [
        parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                        ofproto.OFPCML_NO_BUFFER)
]
inst = [
        parser.OFPInstructionActions(
                ofproto.OFPIT_APPLY_ACTIONS, actions
        )
]
mod = parser.OFPFlowMod(
        datapath=datapath,
        priority=1,
        match=match,
        instructions=inst
)
datapath.send_msg(mod)
```

Invio Packet-Out

Flow Table 0

PRIORITY	MATCH FIELDS	COOKIE	DURATION	IDLE TIMEOUT	HARD TIMEOUT	INSTRUCTIONS	PACKET COUNT	BYTE COUNT	FLAGS
1	ANY	0	78	0	0	OUTPUT:CONTROLLER	42	3276	0

La regola installata nello switch per mandare pacchetti al controllore

Packet-In Handler

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
  def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    dpid = datapath.id
```

La porta da cui è entrato il pacchetto

Parsing del pacchetto

Learning & Lookup

Invio messaggio Packet-Out

Parsing del pacchetto

```
pkt = packet.Packet(msg.data)
eth = pkt.get_protocol(ethernet.ethernet)

dst = eth.dst
src = eth.src
```

Parsing intestazione ethernet

```
self.mac_to_port[dpid][src] = in_port


if dst in self.mac_to_port[dpid]:
    out_port = self.mac_to_port[dpid][dst]
else:
    out_port = ofproto.OFPP_FLOOD
```

Learning

```
pkt = packet.Packet(msg.data)
eth = pkt.get_protocol(ethernet.ethernet)

dst = eth.dst
src = eth.src
```

Parsing intestazione ethernet

Salva MAC sorgente

```
self.mac_to_port[dpid][src] = in_port
```

```
if dst in self.mac_to_port[dpid]:
    out_port = self.mac_to_port[dpid][dst]
else:
    out_port = ofproto.OFPP_FLOOD
```

Lookup

```
pkt = packet.Packet(msg.data)
eth = pkt.get_protocol(ethernet.ethernet)

dst = eth.dst
src = eth.src
```

Parsing intestazione ethernet

Salva MAC sorgente

```
self.mac_to_port[dpid][src] = in_port
```

```
if dst in self.mac_to_port[dpid]:
    out_port = self.mac_to_port[dpid][dst]
else:
    out_port = ofproto.OFPP_FLOOD
```

Trova MAC destinazione

Invio messaggio Packet-Out

```
actions = [
    parser.OFPActionOutput(
      out_port
    )
 ]

 out = parser.OFPPacketOut(
    datapath=datapath,
    buffer_id=msg.buffer_id,
    in_port=in_port,
    actions=actions,
    data= msg.data)
 datapath.send_msg(out)
```

Azione (porta di uscita oppure FLOOD)

Il pacchetto

Esecuzione esercizio 1

Esecuzione del test:

$ sudo mn --mac --topo single,3 --controller remote (-› a stella con 3 host)

$ ryu-manager sdn-labs/3_switch/switch1.py flowmanager/flowmanager.py

Flowmanager @: http://localhost:8080/home/index.html

Per vedere se lo switch funziona: $ mininet/util/m h2 tcpdump –enl

(Python syntax check: $ python3 –m py_compile ‹nome_programma›.py)

Hands-on

1.3

Aggiungere una scorciatoia

Se il pacchetto è instradato verso una porta specifica (non in flooding), allora aggiungere una nuova regola allo switch.
La nuova regola instrada senza passare dal controller tutti i prossimi pacchetti tra la **stessa sorgente** e la **stessa destinazione**.

Testare lo script:

- Su una rete *a stella* con 3 nodi
- Su una rete *lineare* con 3 nodi